

习题五实验报告

一、实验目的

1. 学习如何使用哈希表 (unordered_map) 进行计数和查找。
2. 了解如何在一个组合问题中利用位运算来简化逻辑。
3. 通过实例练习深度理解循环与条件语句的使用。

二、实验原理

本题的主要任务是对一个包含 0 和 1 的二维数据矩阵进行处理，通过位运算来优化问题的解决。首先，我们将矩阵的每一行转换为一个二进制数，再通过哈希表来统计每种二进制模式出现的次数。判断每一行是否符合条件时，我们只需要检查该行中 0 的个数的奇偶性是否与给定参数 K 的奇偶性一致。

三、主要算法步骤

1. **输入：**
 - 输入三个整数 N (矩阵的行数), M (矩阵的列数) 和 K (用于判断的条件)。
 - 输入一个 N 行 M 列的矩阵，每个元素为 0 或 1。
2. **数据结构选择：**
 - 使用哈希表 (unordered_map) 存储每种行的二进制模式及其出现次数。二进制模式通过将每一行的 0 和 1 转化为一个整数来表示。
3. **位运算优化：**
 - 对于矩阵中的每一行，通过左移操作和加法操作，将矩阵中的每一行转换成对应的二进制数。这样我们可以用一个整数来表示整个行，从而减少存储空间。
4. **判断行是否符合条件：**
 - 计算该行中 0 的个数 ct, 如果 ct 与 K 的奇偶性相同, 则认为该行符合条件, 并将该行的二进制模式记录在哈希表中。
5. **查找最大出现次数：**
 - 遍历哈希表中的所有记录，找出出现次数最多的行，并输出该最大次数。
6. **复杂度分析：**该程序的时间复杂度为 $O(N * M)$ ，主要来源于输入二进制数、计算每行二进制值以及哈希表的插入和查找操作。空间复杂度为 $O(N)$ ，主要由哈希表中存储的二进制模式的数量决定。在最坏情况下，每一行的二进制模式都不相同，因此哈希表的大小为 N，所以整体最坏空间复杂度为 $O(N)$ 。

四、伪代码

Input N, M, K

Initialize empty hash table mp

For i = 0 to N-1:

 Initialize binary value x = 0

 Initialize count of 0s ct = 0

 For j = 0 to M-1:

 Input element a

 x = x << 1 // Shift left to make space for the new bit

 x += a // Add the current element a to the binary value

 If a == 0, increment ct

 If $K \geq ct$ and $ct \% 2 == K \% 2$:

 If x is not in mp, set mp[x] = 1

```
        Else increment mp[x]
Initialize maxct = 0
For each entry (x, count) in mp:
    maxct = max(maxct, count)
Output maxct
```

五、实验结果

假设输入如下：

```
4 3 1
1 0 1
1 0 0
0 1 1
1 0 1
```

其中：

- $N = 4$, $M = 3$, $K = 1$ 。
- 输入的矩阵为：
- 1 0 1
- 1 0 0
- 0 1 1
- 1 0 1

输出为：

2

这是因为 1 0 1 这一行出现了两次，并且满足条件（ $K = 1$ ，且该行中 0 的个数是 1，与 1 的奇偶性匹配）。

六、总结

通过这次实验，我学习了如何使用哈希表进行计数并结合位运算优化数据存储。该方法不仅能够高效处理每一行的数据，还能够快速查找出现频率最高的行。使用位运算将矩阵行转换为二进制数，显著减少了内存的使用，并提升了计算效率。这种思路对于其他类似的二进制模式计数问题 also 具有很高的参考价值。